

TRINITY COLLEGE · SEPTEMBER 28, 2026

AI Integration

Structured output, evaluation, and the specification

Ken Kousen · CPSC 415
Monday 1:30–4:10 · MC-225

Where we are

Last week

Your program called the model
You wrote intent before code
You verified with the bill

This week

Your program asks for data
Five cases say whether it worked
You write the spec before code
Teams form



Two chat clients from the tag, on screen, first.

Same client, a different address

```
export CHAT_BASE_URL=http://localhost:11434/v1
export CHAT_API_KEY=ollama                # any string; nothing checks it
export CHAT_MODEL=gemma4:12b-mlx
python3 chat.py "In one sentence, what is a context window?"
```

- No key, no bill, no Activity entry. The cost was the hardware.
- Same request shape. Ollama, LM Studio, and llama.cpp all serve it.
- Slower, and weaker on some questions. Watch for which.

A prompt is an interface, not a wish

Pattern	What it does	Costs
Role	"You route support messages." Sets the frame	Almost nothing
Delimiters	Fences or tags around the data	Almost nothing
Few-shot	Two or three worked examples	Input tokens every call
Chain of thought	"Think step by step"	Output tokens; can hurt on simple tasks

Reasoning models already think before answering. Asking them to think more mostly buys tokens.

Ask for data, not prose

Reply with one JSON object and nothing else:

```
{"category": "billing" | "technical" | "sales" | "unknown",  
  "urgency": "low" | "medium" | "high",  
  "reason": "one short sentence"}
```

Use "unknown" when you cannot tell. Never guess.

- The shape is in the system prompt. The message is the user turn.
- `temperature 0`: you want the same answer for the same input.
- `unknown` is an allowed answer. Without it, the model invents one.

The reply is text until you check it

```
start, end = text.find("{", text.rfind("}")
obj = json.loads(text[start:end + 1])
if obj.get("category") not in CATEGORIES:
    raise ValueError(...)
```

- Models wrap JSON in fences, add a sentence, or return nothing.
- Providers offer `response_format` with a schema. Some models honor it; some ignore it. Your parser is the guarantee.
- A reply you cannot parse is a **failed case**, not a crash.

Five cases are an eval

#	Message	Expect	There to catch
1	Charged twice this month	billing	the easy case
2	App crashes on large PDF	technical	the easy case
3	Discount for 50 seats?	sales, low	urgency judgment
4	Payment failed, can't log in	billing or technical	ambiguity
5	asdf qwerty %%%	unknown	fabrication

Case 5 is the one you will want to skip. It is the one that matters.

When a case fails, which is wrong?

The model

Returned prose
Invented a category
Ran out of tokens

The case

Expected low where medium is
defensible
Ambiguous message, one accepted
answer



Say which kind it was. Do not edit the case to make the model pass.

Chain stage 2: the spec

Stage	Artifact	Who writes	Who approves
Plan	<code>intent/*.md</code>	Agent, from the interview	You
Design	<code>spec.md</code>	Agent, from the intent	You, against the intent
Build	<code>plan.md</code>	Week 4	

If the spec and the intent disagree, the intent wins until you change the intent.

Two decisions per component

Language

Which, and **why**

The alternative you considered

What it cost you: Java has no stdlib
JSON

Model

Which, and **why**

What you compared it against

On which real task, with what result

Today's eval on two models is the model comparison. Write down what happened; that is the "why."

Live: intent to spec

Read `intent/chat-client.md` and write `spec.md` from the template.
One component. Fill in both design decisions with the alternative considered. Number the behaviors. Do not write code.

- Read it against the intent. Find the thing it made up.
- Correct it in the file. Commit "Approve spec."

Teams

- Four teams of two or three.
- Both team projects, or re-form in Week 8.
- Every member explains every part. Checked at the presentation and the Defense.



Ten minutes: form, name, and write three conventions in a shared note.

Team Project 1: Multimodal Web App

- Generates images, displays them, critiques them, **plus** vision or audio.
- Due **November 2** with a ten-minute presentation.
- Team intent due **October 5**. Spec and plan October 19. Reviewed PRs October 26.
- Chain and code weigh about the same. Code with no chain: half credit at most.

Sheet: [assignments/team-project-1.md](#)

Individual Portfolio: the profile site

- A site with a chatbot that answers "should we work together?" **only from your profile**.
- It **refuses to fabricate**. Ten adversarial questions test that.
- Profile, "not yet" inventory, grounding prompt, test set, site. The site is the smallest part.
- `intent/` and `spec.md` due **October 5**. Final December 14. The Defense is over this repo.

Sheet: [assignments/portfolio.md](#) · Reference: github.com/kousen/kousenit

Lab: spec, client, eval, second model

- 1 Template repo, `CLAUDE.md` lines
- 2 Short interview, approve intent
- 3 Spec from intent, correct one thing
- 4 Build; five cases; run
- 5 `max_tokens` 400, run again
- 6 Second model; local if you have one

weeks/03/lab.md · 45 minutes · Prof. Kousen circulates

Save evidence of the work

```
git add .  
git commit -m "Classifier with five-case eval, two models"  
git tag week03-submitted  
git push -u origin main  
git push origin week03-submitted
```

CHECKS.md : one row per model per run. README.md : the five cases and why, the comparison, one spec correction, one line of code.

Before next Monday

- **October 5, 1:30:** Week 3 submission; portfolio `intent/` and `spec.md` ; team intent for TP1.
- Enterprise seat: sign in if the email came; setup walkthrough next week.
- Week 4: retrieval-augmented generation, and `plan.md` .

Before you leave

- What is case 5 there to catch?
- When one model fails a case and another passes, what do you write down?
- What are the two decisions every component in a spec must carry?

Sources and next steps

- Ken Kousen, *Claude Code: Up and Running*: chapters on structured output and evaluation, via Moodle.
- [Artifact-chain template](#): `spec.md` .
- [The AI-Native SDLC Playbook](#): the Design stage.
- [OpenRouter structured outputs](#) · [Ollama OpenAI compatibility](#)
- Reference classifier and eval rehearsed September 22 on `minimax/minimax-m3` , `xiaomi/mimo-v2.5` , and a local `gemma4:12b-mlx` .

Prepared September 22, 2026.